降りタイマー (oriTimer) 詳細設計書

> **この設計書について**
>
> 詳細設計書とは「プログラムをどう作るか」を書いたドキュメントです。
> 基本設計書（何を作るか）の次のステップにあたり、
> 開発者がコードを書くために必要な情報をすべて記載します。
>
> 主に以下の観点で構成されます。

セクション	書く内容	なぜ必要か
1. システム概要	アプリ全体の目的と構成	新メンバーが全体像を把握するため
2. アーキテクチャ	技術選定と設計方針	「なぜこう作ったか」を残すため
3. ディレクトリ構成	ファイルの配置ルール	どこに何があるか迷わないため
4. データモデル	データの型と構造	APIとの整合性を保つため
5. 状態管理	Providerの一覧と役割	データの流れを明確にするため
6. 画面設計	各画面のUI構成と動作	実装の仕様を明確にするため
7. API連携	エンドポイントと通信仕様	サーバーとの契約を明文化するため
8. ジオフェンス仕様	ジオフェンスの設定値と動作	プラットフォーム差異を明確にするため
9. ビルド手順	開発環境のセットアップ	「動かない」を防ぐため

---

1. システム概要

> **このセクションの目的**：プロジェクトに初めて触れる人が「これは何のアプリか」を理解できるようにする。

1.1 アプリの目的

東京の電車駅を選択し、その駅の半径1000m圏内に入ったときにプッシュ通知とループバイブレーション(ユーザーが停止するまで継続)で知らせる、乗り越し防止モバイルアプリケーション。
外部APIサーバーから路線・駅データを取得し、OSネイティブのジオフェンス機能を利用して位置ベースのアラームを実現する。

1.2 主な機能

機能	
路線・駅一覧表示	外部APIから取得した東京の全路線・駅を路線ごとにグループ
駅名検索	駅名を前方一致で検索し、ダイアログから路線を選んで該当
選択駅ジャンプ / 同名駅循環ジャンプ	選択駅を含む最初の路線へ即座にジャンプ(📍ボタン)。同
駅選択	一覧から降りたい駅を1つ選択する
ジオフェンス監視	選択駅の座標に半径1000mのジオフェンスを登録し、バッ
降車通知	圏内進入時にプッシュ通知とループバイブレーション(最大
監視状態の永続化	SharedPreferencesに監視ON/OFFを保存し、アプリ再起動
パーミッション管理	位置情報(常に許可)と通知パーミッションを順次要求する
複数目的地登録(マルチゴール)	最大10件の目的地を登録し、それぞれのジオフェンスを同時
ルートパターン管理	2件以上のマルチゴールをパターンとして保存・一覧表示・選択
マルチゴール1件時の単一選択同等表示	マルチゴールが1件になったとき、effectiveStationName に

1.3 動作環境

項目	値
フレームワーク	Flutter 3.x (Dart 3.x)
対応OS	Android / iOS

項目	値
画面方向	縦固定
外部APIサーバー	http://toyohide.work (POST通信)

---

2. アーキテクチャ︵設計方針︶

> **このセクションの目的**：﹁なぜこのライブラリを使うのか﹂﹁どういうルールでコードを書くのか﹂を明文化する。後から参加したメンバーが勝手に違う方針で書くのを防ぐ。

2.1 技術スタック

```
┌──────────────────────────────────────────────────────────────┐
│                          UI層                                 │
│  Widget (HookConsumerWidget / StatelessWidget)                 │
│  flutter_hooks (useXxx でライフサイクル管理)                    │
├──────────────────────────────────────────────────────────────┤
│                        状態管理層                             │
│  Riverpod (@riverpod アノテーション方式)                        │
│  コード生成: riverpod_generator + build_runner                  │
├──────────────────────────────────────────────────────────────┤
│                       データモデル層                           │
│  freezed (イミュータブルモデル)                                 │
│  json_serializable (JSON変換)                                  │
├──────────────────────────────────────────────────────────────┤
│                        通信層                                  │
│  http パッケージ (REST API通信)                                 │
│  flutter_dotenv (.env ファイルによる環境変数管理)               │
├──────────────────────────────────────────────────────────────┤
│                     ジオフェンス・通知層                        │
│  native_geofence (OSネイティブジオフェンスの利用)               │
│  flutter_local_notifications (プッシュ通知)                     │
│  vibration (ループバイブレーション制御)                         │
│  flutter_volume_controller (音楽ストリーム音量制御)             │
│  permission_handler (パーミッション要求)                        │
├──────────────────────────────────────────────────────────────┤
│                       永続化層                                 │
│  shared_preferences (監視状態・マルチゴールの端末保存・復元)    │
│  SharedPreferencesService (永続化操作をまとめたユーティリティ)  │
├──────────────────────────────────────────────────────────────┤
│                      スクロール制御層                          │
│  scrollable_positioned_list (インデックス指定スクロール)        │
└──────────────────────────────────────────────────────────────┘

              ↕ HTTP POST (JSON)

┌──────────────────────────────────────────────────────────────┐
│                外部APIサーバー (toyohide.work)                  │
└──────────────────────────────────────────────────────────────┘
```

2.2 設計ルール

ルール	説明
Widgetの基底クラス	Providerを使う画面は `HookConsumerWidget` ｀使わない場合は `Statele` を使う
状態管理	`@riverpod` アノテーション + コード生成 ゜手書きProviderは使わない
モデル	`@freezed` で定義 ゜イミュータブル︵不変︶を保証する
API通信	Provider内で `HttpClient.post()` を呼び出す ゜Widgetから直接APIを呼
ジオフェンスコールバック	トップレベル関数として定義する︵Flutterのバックグラウンド実行の制
テーマ	ダークテーマ統一 ゜MaterialApp の `themeMode: ThemeMode.dark` を設定

2.3 カラーパレット

> **補足** ： 設計書にカラーコードを載せておくと｀新しい画面を追加するときに統一感を保てる ゜

用途	値	説明
テーマ	`ThemeData.dark()`	Flutter標準ダークテーマ
選択状態︵アクティブ︶	`Colors.yellow`	選択中の駅インジケーター｀監視中ボタン｀パ
選択状態︵非アクティブ︶	`Colors.white` (枠のみ)	未選択の駅インジケーター
アクセント	`Colors.blueAccent`	﹁監視中﹂ラベル文字色
テキスト︵サブ情報︶	`Colors.white60`	緯度·経度などの補足情報

---

3. ディレクトリ構成

> **このセクションの目的** ： ﹁新しいファイルをどこに置けばいいか﹂を明確にする ゜

```
lib/
├── main.dart                          … アプリのエントリーポイント
├── const/
│   └── const.dart                     … アプリ全体の定数（バイブレーションパターン等）
├── screens/
│   ├── home_screen.dart               … ホーム画面（唯一のメイン画面）
│   ├── components/
│   │   ├── multi_goal_display_alert.dart      … マルチゴール一覧ダイアログ（S-03）
│   │   ├── multi_goal_setting_alert.dart      … マルチゴール設定ダイアログ（S-04）
│   │   └── pattern_route_display_alert.dart   … パターン一覧ダイアログ（S-05）
│   └── parts/
│       ├── error_dialog.dart          … エラーダイアログ
│       └── oritimer_dialog.dart       … フルスクリーンダイアログ共通ラッパー
├── model/
│   └── tokyo_train_model.dart         … 路線・駅のデータモデル（freezed）
├── controllers/
│   ├── controllers_mixin.dart         … Provider参照をまとめたMixin
│   ├── app_param/
│   │   ├── app_param.dart             … アプリ状態のProviderとNotifier
│   │   ├── app_param.freezed.dart     … 自動生成: Freezedクラス
│   │   └── app_param.g.dart           … 自動生成: Riverpod Provider
│   └── _get_data/
│       └── tokyo_train/
│           ├── tokyo_train.dart       … 路線・駅データのProviderとNotifier
│           ├── tokyo_train.freezed.dart … 自動生成: Freezedクラス
│           └── tokyo_train.g.dart     … 自動生成: Riverpod Provider
├── data/
│   └── http/
│       ├── client.dart                … HttpClientクラス、環境変数設定
│       └── path.dart                  … APIエンドポイントのEnum定義
├── extensions/
│   └── extensions.dart                … DateTime, String, BuildContext の拡張関数
├── utility/
│   ├── utility.dart                   … Utilityクラス（エラー表示等）
│   ├── functions.dart                 … トップレベル関数（geofenceCallback・getTrainIndicesFo
│   └── shared_preferences_service.dart  … SharedPreferences操作をまとめたユーティリティ
└── assets/
    ├── images/
    │   ├── ic_launcher.png            … アプリアイコン
```

```
│   └── mezamashi.png              … スプラッシュ画面用画像
└── .env                           … 環境変数ファイル（APIベースURLなど）
```

配置ルール

ディレクトリ	置くもの
`const/`	アプリ全体で共有する定数︵バイブレーションパターン等︶
`screens/`	ページ全体を構成するWidget
`screens/components/`	ダイアログなどの画面コンポーネント︵複数画面から利用されるWidget︶
`screens/parts/`	ダイアログ共通ラッパーなどの小部品
`model/`	freezedモデルのみ
`controllers/`	`@riverpod` Provider｡API通信ロジック
`data/http/`	HTTPクライアント｡APIパス定義
`extensions/`	既存クラスへの拡張関数
`utility/`	汎用ユーティリティ︵エラー表示｡SharedPreferences操作｡トップレベル

---

4. データモデル定義

> **このセクションの目的**：APIから受け取るデータの﹁型﹂を明確にする。サーバー側とクライアント側で﹁このフィールドは何型か﹂を合意するための仕様書になる。

4.1 TokyoStationModel︵駅情報︶

用途：各駅の情報を保持する。路線情報︵TokyoTrainModel︶に含まれる。

フィールド名	**JSON キー**	**Dart型**	必須	説明
id	id	`String`	Yes	駅の一意識別子

フィールド名	JSON キー	Dart型	必須	説明
stationName	stationName	`String`	Yes	駅名（例: "渋谷"）
address	address	`String`	Yes	駅の住所
lat	lat	`double`	Yes	緯度（例: 35.6580）
lng	lng	`double`	Yes	経度（例: 139.7016）

freezedクラス定義（概要）:

```
@freezed
class TokyoStationModel with _$TokyoStationModel {
  const factory TokyoStationModel({
    required String id,
    required String stationName,
    required String address,
    required double lat,
    required double lng,
  }) = _TokyoStationModel;

  factory TokyoStationModel.fromJson(Map<String, dynamic> json)
      => _$TokyoStationModelFromJson(json);
}
```

4.2 TokyoTrainModel（路線情報）

用途：路線情報と、その路線に属する駅のリストを保持する。

フィールド名	JSON キー	Dart型	必須	説明
trainNumber	trainNumber	`int`	Yes	路線の識別番号
trainName	trainName	`String`	Yes	路線名（例: "山手線"）
station	station	`List<TokyoStationModel>`	Yes	路線に属する駅のリスト

freezedクラス定義（概要）:

```
@freezed
class TokyoTrainModel with _$TokyoTrainModel {
  const factory TokyoTrainModel({
    required int trainNumber,
    required String trainName,
    required List<TokyoStationModel> station,
  }) = _TokyoTrainModel;

  factory TokyoTrainModel.fromJson(Map<String, dynamic> json)
      => _$TokyoTrainModelFromJson(json);
}
```

4.3 AppParamState（アプリ状態）

用途：アプリの動作状態を保持する。SharedPreferencesと連携して、アプリ再起動後も状態を復元する。

フィールド名	Dart型	初期値	説明
isSetStation	`bool`	`false`	ジオフェンス監視中かどうかのフラグ（SharedP
selectedMultiNumber	`int`	`-1`	マルチゴール設定ダイアログで選択中のスロット -1
selectedStationName	`String`	`''`	マルチゴール設定ダイアログで選択中の駅名
selectedPatternDispString	`String`	`''`	パターン一覧ダイアログで選択中のパターン表示 品川 → 蒲田"）。未選択時は空文字

freezedクラス定義（概要）:

```dart
@freezed
class AppParamState with _$AppParamState {
  const factory AppParamState({
    @Default(false) bool isSetStation,
    @Default(-1) int selectedMultiNumber,  // 未選択状態を -1 で表現
    @Default('') String selectedStationName,
    @Default('') String selectedPatternDispString,
  }) = _AppParamState;
}
```

SharedPreferencesキー（AppParam管理分）:

キー名	型	保存タイミング	削除タイミング
`'isSetStation'`	`bool`	目のアイコンタップ時	停止ボタン（×）タップ時

> **補足**：停止時は `setBool(false)` ではなく `remove()` でキーを削除する設計にしている。`getBool()` が `null` を返したときに `?? false` でデフォルト値を適用するためである。

SharedPreferencesキー（HomeScreen / SharedPreferencesService 管理分）:

キー名	型	保存タイミング	
`'selectedStation'`	`String`（JSON）	駅タップ時（`_saveSelectedStation()`）	停止ボタ
`'multiGoal_N'`	`String`	マルチゴール登録時	マルチゴ
`'multiGoalLat_N'`	`double`	マルチゴール登録時	マルチゴ
`'multiGoalLng_N'`	`double`	マルチゴール登録時	マルチゴ

> **補足**：`N` は 0~9 のスロット番号。`multiGoal_N` が駅名、`multiGoalLat_N` / `multiGoalLng_N` が座標。座標はアプリ再起動後のジオフェンス復元に使用する。`SharedPreferencesService` クラス（`utility/shared_preferences_service.dart`）がこれらのキーの読み書き・削除をすべて担当する。

> **補足**：選択駅は `TokyoStationModel.toJson()` で JSON 文字列に変換して保存し、復

> 元時は `TokyoStationModel.fromJson()` でパースする。監視状態(`isSetStation`)は `AppParam` が管理するのに対し、選択駅・マルチゴール情報は `SharedPreferencesService` 経由で扱う設計にしている。

4.4 TokyoTrainState(路線・駅データ状態)

用途：APIから取得した全路線・駅データと、アクセス効率化のためのマップを保持する。

フィールド名	**Dart型**	
tokyoTrainList	`List<TokyoTrainModel>`	全路線リスト
tokyoTrainMap	`Map<String, TokyoTrainModel>`	路線名をキー
tokyoStationTokyoTrainModelListMap	`Map<String, List<TokyoStationModel>>`	駅名をキーと

> **補足**：`tokyoTrainMap` と `tokyoStationTokyoTrainModelListMap` は、リストを毎回走査するO(n)のコストを回避し、O(1)でデータにアクセスするために用意している。

---

5. 状態管理設計(Provider一覧)

> **このセクションの目的**：アプリ内でデータがどこで取得され、どこで使われるかを明確にする。「このデータはどのProviderから取ればいいの？」という疑問に答える。

5.1 Provider一覧

```
┌──────────────────────────────────────────────────────────────────┐
│                              Provider一覧                          │
├──────────────────────────┬──────────────────────┬──────────────────┤
│ Provider名               │ 種別                 │ 返却型           │
├──────────────────────────┼──────────────────────┼──────────────────┤
│ appParamProvider         │ NotifierProvider     │ AppParamState    │
│ tokyoTrainProvider       │ NotifierProvider     │ TokyoTrainState  │
└──────────────────────────┴──────────────────────┴──────────────────┘
```

5.2 各Providerの詳細

appParamProvider

```
種別:        NotifierProvider（ユーザー操作で値が変わる）
返却型:      AppParamState
初期値:      AppParamState() → isSetStation: false, selectedMultiNumber: -1, selectedStatio
Notifier:   AppParam

操作メソッド:
  setIsSetStation({required bool flag})
    → isSetStation を更新する（監視開始時: true、停止時: false）
    → 同時に SharedPreferences に保存する（_persistFlag を内部で呼ぶ）
    → flag=true のとき: prefs.setBool('isSetStation', true)
    → flag=false のとき: prefs.remove('isSetStation')

  loadFromPrefs()  ← アプリ起動時に呼ぶ
    → SharedPreferences から 'isSetStation' を読み込む
    → 値が存在すれば isSetStation: true として state を更新する
    → 値がなければ（null）isSetStation: false のまま（デフォルト）

  setSelectedMultiNumber({required int number})
    → selectedMultiNumber を更新する（マルチゴール設定ダイアログで次の空きスロットを設定）

  setSelectedStationName({required String name})
    → selectedStationName を更新する（マルチゴール設定ダイアログで駅選択時）

  saveMultiGoalEntry() → bool
    → selectedMultiNumber と selectedStationName が有効な場合、SharedPreferencesService 経由で
    → 保存成功時: true を返す。未選択の場合: false を返す

  deleteMultiGoalEntry({required int number})
    → SharedPreferencesService.deleteMultiGoalEntry() を呼び出してエントリを削除する

  loadAllMultiGoalEntries() → Map<int, String>
    → SharedPreferencesService.loadAllMultiGoalEntries() を呼び出して全エントリを返す

  setSelectedPatternDispString({required String str})
    → selectedPatternDispString を更新する（パターン一覧ダイアログでCircleAvatarタップ時）

使用箇所:    HomeScreen（ジオフェンス監視状態の管理・表示・復元）
             MultiGoalDisplayAlert（マルチゴール一覧・削除・新規設定ダイアログ起動）
             MultiGoalSettingAlert（マルチゴール設定・駅選択）
```

```
                PatternRouteDisplayAlert（パターン選択状態の管理）
```

tokyoTrainProvider

```
種別:         NotifierProvider（アプリ起動時にデータ取得）
返却型:       TokyoTrainState
初期値:       TokyoTrainState() → 全フィールドが空

Notifier:   TokyoTrain
設定:         keepAlive: true（アプリ全体でデータを保持）

操作メソッド:
  getAllTokyoTrain()
    → fetchAllTokyoTrainData() を呼び出し、結果をstateに格納する
    → 取得したリストからtokyoTrainMapとtokyoStationTokyoTrainModelListMapも構築する

  fetchAllTokyoTrainData()
    → HTTP POST /BrainLog/api/getTokyoTrainStation
    → レスポンスJSONをList<TokyoTrainModel>に変換して返す
    → エラー時はUtility.errorLog()で記録し、空リストを返す

使用箇所:     HomeScreen（路線・駅一覧の表示）
              AppRoot（アプリ起動時の初期データ取得）
```

5.3 ControllersMixin

```
// 全Widgetで共通して使えるProviderアクセスのMixin
// ConsumerStatefulWidget にも対応（on ConsumerState or ConsumerWidget）
mixin ControllersMixin<T extends ConsumerStatefulWidget> on ConsumerState<T> {
  // AppParam
  AppParamState get appParamState   → ref.watch(appParamProvider)
  AppParam get appParamNotifier     → ref.read(appParamProvider.notifier)

  // TokyoTrain
  TokyoTrainState get tokyoTrainState   → ref.watch(tokyoTrainProvider)
  TokyoTrain get tokyoTrainNotifier     → ref.read(tokyoTrainProvider.notifier)
}
```

> **補足：** `ref.watch()` と `ref.read()` の使い分けは重要。
>
> - `ref.watch()`：値が変わったとき自動でWidgetが再描画される（表示用）
> - `ref.read()`：1回だけ読み取る（ボタンのonPressed等、アクション内での操作用）

---

6. 画面設計

> **このセクションの目的：** 各画面の構成要素、表示ルール、操作時の振る舞いを定義する。開発者はこのセクションを見ればUIを実装できる。

6.1 エントリーポイント (main.dart)

```
初期化処理（AppRoot._initState）：
  1. WidgetsFlutterBinding.ensureInitialized()
  2. 画像キャッシュの設定
       - PaintingBinding.instance.imageCache.maximumSize = 150（最大150枚）
       - PaintingBinding.instance.imageCache.maximumSizeBytes = 80 << 20（最大80MB）
  3. .envファイルの読み込み（flutter_dotenv）
  4. ProviderScope でアプリ全体をラップ
  5. AppRoot を起動

AppRoot (HookConsumerWidget):
  - initState相当の処理:
      tokyoTrainNotifier(ref).getAllTokyoTrain()  ← 起動時に駅データを取得

MyApp (StatelessWidget):
  - MaterialApp の設定:
      - テーマ: ThemeData.dark()
      - themeMode: ThemeMode.dark
      - ホーム画面: HomeScreen
      - デバッグバナー: 非表示
      - ローカライゼーション: 英語・日本語
```

6.2 ホーム画面 (HomeScreen)

Widgetの種別: ConsumerStatefulWidget（with ControllersMixin）

ローカル状態·コントローラー:

- `TokyoStationModel? _selected` — 選択中の駅情報
- `bool _permissionsGranted` — パーミッション付与状態
- `int _destinationOccurrenceIndex` — 同名駅循環ジャンプで現在表示中の路線インデックス︵駅選択時に 0 リセット︶
- `Map<int, String> _multiGoalMap` — 登録済みマルチゴールのスロット番号→駅名マップ
- `ItemScrollController _itemScrollController` — インデックス指定スクロール用 ︵scrollable_positioned_list︶
- `TextEditingController _searchController` — 検索フォームのテキスト管理

build() 内で算出するローカル変数:

- `String? effectiveStationName` — 実効的な選択駅名 ˚`_selected?.stationName` が優先 ˚null かつ `_multiGoalMap.length == 1` のとき `_multiGoalMap.values.first` を使用 ˚それ以外は null
  - ︾選択中︿テキスト: `effectiveStationName ?? (_multiGoalMap.length > 1 ? "複数" : "(未選択)")`
  - BBBジャンプボタン: `effectiveStationName != null` のとき有効化

初期化処理︵initState︶:

- `_initPlugins()` と `_loadMultiGoals()` を呼び出す:
  i. FlutterLocalNotificationsPlugin の初期化
  ii. NativeGeofenceManager の初期化
  iii. パーミッション状態の確認 → `_permissionsGranted` を更新
  iv. `appParamNotifier.loadFromPrefs()` → SharedPreferences から監視状態 ︵`isSetStation`︶を復元する
  v. SharedPreferences から `'selectedStation'` を読み込み → `TokyoStationModel.fromJson()` でパース → `setState(() => _selected = station)` で復元する
  vi. `_restoreMultiGoalGeofences()` → SharedPreferencesService から全マルチゴールエントリと座標を読み込み ˋジオフェンスを再登録する

画面レイアウト:

```
┌──────────────────────────────────────────────────────────────────┐
│  AppBar                                                          │
│  降りタイマー                    [🔒]        [  ]        [×]  │
│                          パーミッション要求   監視開始/中   監視停止 │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│  [  駅名を検索...                          [×]  ] [検索]         │  ← F-11
│                                                                  │
│  ┌──────────────────────────────────────────────────────────┐   │
│  │  現在の選択駅: 渋谷駅                    ★ 監視中          │ │
│  └──────────────────────────────────────────────────────────┘   │
│                                                                  │
├──────────────────────────────────────────────────────────────────┤
│  ┌── 山手線 ──────────────────────────────────────────────┐     │
│  │  ●  渋谷   ← 選択中（黄色の丸）                          │  │
│  │     緯度: 35.658, 経度: 139.701                            │  │
│  │  ○  新宿   ← 未選択（白枠の丸）                          │  │
│  │     緯度: 35.689, 経度: 139.700                            │  │
│  └──────────────────────────────────────────────────────────┘    │
│  ┌── 中央線 ──────────────────────────────────────────────┐     │
│  │  ...                                                     │  │
│  └──────────────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────────────┘
```

AppBarボタンの実装仕様:

ボタン	アイコン	表示条件	
パーミッション要求	`Icons.security`	常時	`_permissions` `Colors.yello` デフォルト
監視開始	`Icons.remove_red_eye`	常時	`appParamStat` または `_mult` `Colors.yello` デフォルト
監視停止	`Icons.close`	`_multiGoalMap.length <= 1`	常にデフォル

ボタン	アイコン	表示条件	
監視停止︿メニュー︾	`Icons.more_vert`	`_multiGoalMap.length >= 2`	常にデフォル

現在の選択駅表示:

```
// 表示内容
駅名:     _selected?.stationName ?? '(未選択)'
監視状態: appParamState.isSetStation == true の場合のみ「★ 監視中」を表示

// 「複数」ボタン（ホーム画面のbody Row内）
ElevatedButton(
  onPressed: () {
    OritimerDialog(context: context, widget: MultiGoalDisplayAlert()).then((_) {
      _loadMultiGoals();  // ダイアログを閉じた後に一覧を更新
    });
  },
  child: Text('複数'),
)
```

マルチゴール復元処理 `_restoreMultiGoalGeofences()`:

```
Future<void> _restoreMultiGoalGeofences() async {
  final Map<int, String> entries = await SharedPreferencesService.loadAllMultiGoalEntries(

  for (final MapEntry<int, String> entry in entries.entries) {
    final ({double? lat, double? lng}) location =
        await SharedPreferencesService.loadMultiGoalLocation(number: entry.key);

    if (location.lat == null || location.lng == null) continue;

    final Geofence zone = Geofence(
      id: 'multiGoal_${entry.key}',
      location: Location(latitude: location.lat!, longitude: location.lng!),
      radiusMeters: 1000,
      triggers: {GeofenceEvent.enter},
      iosSettings: IosGeofenceSettings(initialTrigger: true),
      androidSettings: AndroidGeofenceSettings(
        initialTriggers: {GeofenceEvent.enter},
        expiration: Duration(days: 7),
        loiteringDelay: Duration(minutes: 1),
        notificationResponsiveness: Duration(seconds: 10),
      ),
    );
    try {
      await NativeGeofenceManager.instance.createGeofence(zone, geofenceCallback);
    } catch (_) {}
  }
}
```

選択駅ジャンプ / 同名駅循環ジャンプの実装仕様（F-12 / F-13）:

```
// 選択駅名を含む全路線インデックスを返す
// 実体は lib/utility/functions.dart の getTrainIndicesForStation() に定義
// HomeScreen 内では以下のラッパーを通して呼び出す
List<int> _getTrainIndicesForStation(String stationName) =>
    getTrainIndicesForStation(stationName: stationName, trainList: widget.tokyoTrainList)
```

選択駅ジャンプボタン（📍 / `Icons.location_on`）の処理:

```
1. _getTrainIndicesForStation(selected.stationName) で路線インデックスリストを取得
2. リストが空でなければ _destinationOccurrenceIndex = 0 にリセット
3. indices[0] へ _jumpToIndex() でジャンプ
```

同名駅循環ジャンプボタン（↕/ `Icons.swap_vert`）の処理:

```
1. _getTrainIndicesForStation(selected.stationName) で路線インデックスリストを取得
2. リストが空なら何もしない
3. next = (_destinationOccurrenceIndex + 1) % indices.length で次インデックスを計算（循環）
4. _destinationOccurrenceIndex = next に更新
5. indices[next] へ _jumpToIndex() でジャンプ
```

ボタン表示ルール:

- 駅選択中: 📍と↕の両ボタンを表示（onPressed 有効）
- 未選択時: 同じサイズの透明ダミーボタンを表示（レイアウト崩れ防止）

駅選択時のリセット:

```
onTap: () {
  setState(() {
    _selected = element2;
    _destinationOccurrenceIndex = 0;  // 駅が変わったらカウンターをリセット
  });
  _saveSelectedStation(element2);
}
```

駅名検索の実装仕様（`_jumpToIndex()` / 検索フォーム）:

```
// インデックス指定スクロール
void _jumpToIndex(int index) {
  if (!_itemScrollController.isAttached) return;
  _itemScrollController.scrollTo(
    index: index,
    duration: const Duration(milliseconds: 450),
    curve: Curves.easeInOut,
  );
}

// build() 内でインデックスマップを構築
final Map<String, int> firstIndexByTrainName = {};
for (int i = 0; i < widget.tokyoTrainList.length; i++) {
  firstIndexByTrainName[widget.tokyoTrainList[i].trainName] = i;
}
```

検索フォームの構成:

- `Row(TextField + ElevatedButton)` を BBB 位置に配置
- `TextField` の `suffixIcon`: 入力中のみ×ボタンを表示︵`ValueListenableBuilder` で制御︶
- 検索ボタン押下時:
  i. キーボードを閉じる︵`FocusScope.of(context).unfocus()`︶
  ii. `_searchController.text` を取得後クリア
  iii. `tokyoStationTokyoTrainModelListMap.entries` を前方一致でフィルタ
  iv. 結果を `AlertDialog` で表示
  v. 路線 `ListTile` タップ → `Navigator.pop()` → `_jumpToIndex(firstIndexByTrainName[trainName])`

検索ダイアログ内のフラットリスト構造:

```
[{isHeader: true,  stationName: "渋谷駅",    train: null     }]
[{isHeader: false, stationName: "渋谷駅",    train: 山手線   }]
[{isHeader: false, stationName: "渋谷駅",    train: 東急東横線}]
[{isHeader: true,  stationName: "渋谷本町",  train: null     }]
[{isHeader: false, stationName: "渋谷本町",  train: 都営大江戸}]
```

- `isHeader == true` → 駅名テキスト︵太字︶

- `isHeader == false` → `ListTile`（路線名 + Icons.train）

路線·駅リストの実装仕様:

```
Widget: ScrollablePositionedList.builder (scrollable_positioned_list)
  itemScrollController: _itemScrollController
  └── 路線ごとにグループ (ExpansionTile)
        ├── title: 路線名テキスト
        └── 駅リスト: 駅ごとに Row

駅 Row の構成:
  leading: CircleAvatar (radius: 15)
    - 選択中:  backgroundColor: Colors.yellowAccent.withValues(alpha: 0.3)
    - 非選択:  backgroundColor: Colors.black.withValues(alpha: 0.3)
    - onTap:  setState(() => _selected = station) + _saveSelectedStation()
  title:    駅名 (maxLines: 1)
  subtitle: 緯度・経度を縦並びで表示
```

パーミッション要求処理 `_requestPermissions()`:

```
1. permission_handler で locationWhenInUse を要求
2. 付与された場合、locationAlways を要求
3. 付与された場合、notification を要求
4. 全て付与された場合:
     _isPermissionGranted.value = true
5. いずれかが拒否された場合:
     エラーメッセージ (SnackBar等) を表示
```

ジオフェンス登録処理 `_registerSelectedStation()`:

```
前提チェック:
  - _selectedStation.value が null の場合 → エラーメッセージを表示して終了

処理:
  1. NativeGeofenceManager.instance.createGeofence() を呼び出す
       - geofenceId: 'oritimer_geofence'
       - lat: _selectedStation.value!.lat
       - lng: _selectedStation.value!.lng
       - radiusMeters: 500.0
       - triggers: [GeofenceEvent.enter]
       - callback: _geofenceTriggered（トップレベル関数）
       - iOS設定:
           - initialTrigger: true（アプリ起動時にすでに圏内の場合も発火）
       - Android設定:
           - expiration: Duration(days: 7)（7日間有効）
           - loiteringDelay: Duration(minutes: 1)（進入後1分の滞留で発火）
           - responsiveness: Duration(seconds: 10)（応答性: 10秒）
  2. appParamNotifier.setIsSetStation(flag: true) で状態を更新
```

6.3 ジオフェンスコールバック（トップレベル関数）

> **補足**：Flutterでバックグラウンドから関数を呼び出す場合、トップレベル関数（classの外）として定義する必要がある。これはDartのアイソレート（スレッド）の仕組みによる制約。
>
> **ファイル:** `lib/utility/functions.dart`（`home_screen.dart` から抽出済み）。バイブレーションパターンは `lib/const/const.dart` の `kVibrationPattern` / `kVibrationIntensities` 定数を使用する。

```dart
@pragma('vm:entry-point')
Future<void> geofenceCallback(GeofenceCallbackParams params) async {
  WidgetsFlutterBinding.ensureInitialized();
  try { DartPluginRegistrant.ensureInitialized(); } catch (_) {}

  // 1. FlutterLocalNotificationsPlugin の初期化
  final FlutterLocalNotificationsPlugin notifications =
      FlutterLocalNotificationsPlugin();
  await notifications.initialize(/* 省略 */);

  // 2. 音楽ストリームの音量を最大に上げる (Android のみ)
  //    システム UI のスライダーを出さずに変更する
  if (Platform.isAndroid) {
    try {
      await FlutterVolumeController.updateShowSystemUI(false);
      await FlutterVolumeController.setVolume(1.0, stream: AudioStream.music);
    } catch (_) {}
  }

  // 3. プッシュ通知を発火
  await notifications.show(
    id: DateTime.now().millisecondsSinceEpoch ~/ 1000,
    title: '降りる駅アラーム',
    body: params.geofences.map((g) => g.id).join(', '),
    notificationDetails: NotificationDetails(
      android: AndroidNotificationDetails(
        'geofence', 'Geofence',
        importance: Importance.max,
        priority: Priority.high,
        vibrationPattern: Int64List.fromList([0, 600, 100, 600, 100, 600, 100, 1000]),
      ),
      iOS: const DarwinNotificationDetails(),
    ),
  );

  // 4. ループバイブレーション開始 (Android のみ)
  if (Platform.isAndroid) {
    final bool? hasVibrator = await Vibration.hasVibrator();
    if (hasVibrator == true) {
```

```
      await Vibration.vibrate(
        pattern: [0, 600, 100, 600, 100, 600, 100, 1000],
        intensities: [0, 255, 0, 255, 0, 255, 0, 255],  // 最大強度
        repeat: 0,  // 先頭からループ
      );
    }
  }
}
```

バイブレーションパターン︵**v1.1**以降︶**:**

> **定数定義場所:** `lib/const/const.dart` の `kVibrationPattern` / `kVibrationIntensities` ゜`geofenceCallback` 内で直接リテラルを書かず、これらの定数を参照する ゜

インデックス	値(ms)	強度 (0-255)	意味
0	0	0	開始直後︵待機なし︶
1	600	255	600ms バイブレーション︵最大強度︶
2	100	0	100ms 停止
3	600	255	600ms バイブレーション︵最大強度︶
4	100	0	100ms 停止
5	600	255	600ms バイブレーション︵最大強度︶
6	100	0	100ms 停止
7	1000	255	1000ms バイブレーション︵最大強度︶
︵ループ︶	`repeat: 0`	-	index 0 に戻って繰り返す

> **補足 :** `repeat: 0` はAndroidの `VibrationEffect.createWaveform(timings, amplitudes, repeat)` の第3引数に相当する ゜`0` を指定するとインデックス0から繰り返し再生される ゜`Vibration.cancel()` を呼ぶまで停止しない ゜
>
> `intensities` パラメータは Android 8.0 (API 26) 以上で有効 ゜それ以下のバージョンで

> はパターンのみ︵強度はデフォルト︶で動作する゜

バイブレーション停止·選択駅クリア処理︵`_removeAllGeofences()`︶:

```
Future<void> _removeAllGeofences() async {
  // 1. OSのジオフェンスをすべて解除
  await NativeGeofenceManager.instance.removeAllGeofences();

  // 2. ループ中のバイブレーションを即時停止（Android のみ）
  if (Platform.isAndroid) {
    await Vibration.cancel();
  }

  // 3. 選択駅を SharedPreferences から削除
  final SharedPreferences prefs = await SharedPreferences.getInstance();
  await prefs.remove('selectedStation');

  // 4. 画面の選択駅表示をリセット
  if (mounted) {
    setState(() => _selected = null);
  }
}

// dispose() 内（画面が破棄されるときの安全処理）
@override
void dispose() {
  if (Platform.isAndroid) {
    Vibration.cancel();
  }
  super.dispose();
}
```

選択駅の保存処理︵`_saveSelectedStation()`︶:

```
Future<void> _saveSelectedStation(TokyoStationModel station) async {
  final SharedPreferences prefs = await SharedPreferences.getInstance();
  await prefs.setString('selectedStation', jsonEncode(station.toJson()));
}
```

駅タップ時の呼び出し:

```
onTap: () {
  setState(() => _selected = station);  // 画面を即時更新
  _saveSelectedStation(station);        // SharedPreferences に非同期保存
}
```

6.4 マルチゴール一覧ダイアログ（MultiGoalDisplayAlert）

ファイル: `lib/screens/components/multi_goal_display_alert.dart`
Widgetの種別: ConsumerStatefulWidget（with ControllersMixin）

ローカル状態:

- `Map<int, String> _multiGoalMap` — スロット番号→駅名。initState で `appParamNotifier.loadAllMultiGoalEntries()` から読み込む

画面構成:

```
┌──────────────────────────────────────┐
│  multi goal list     [パターン][設定]   │
│ ──────────────────────────────────────  │
│              [パターン登録]              │  ← _multiGoalMap.length >= 2 のときのみ
│ ──────────────────────────────────────  │
│  ① 渋谷駅                               │
│  ② 品川駅                               │
│  ③ 新宿駅                    [  ]       │  ← 最後のゴール（最大スロット番号）のみ
└──────────────────────────────────────┘
```

「パターン」ボタンの処理:

1. `OritimerDialog` で `PatternRouteDisplayAlert` を表示する
2. 閉じたら `_loadMultiGoals()` を呼ぶ

「パターン登録」ボタンの処理（`_multiGoalMap.length >= 2` のときのみ表示）:

1. `_multiGoalMap` をソートして駅名リストを作成する
2. `SharedPreferencesService.saveRoutePattern(name: タイムスタンプ, stations: 駅リスト)`

で保存する
3. `error_dialog(title: '登録しました', content: '駅A → 駅B → 駅C')` で確認を表示する

¬設定˫ ボタンの処理:

1. `appParamNotifier.loadAllMultiGoalEntries()` で現在の登録件数を確認する
2. 10件以上の場合は `error_dialog` を表示して終了する
3. 空きスロット番号︵0~9 で `registered.containsKey(nextSlot)` が false の最小値︶を求める
4. `appParamNotifier.setSelectedMultiNumber(number: nextSlot)` と `setSelectedStationName(name: '')` を呼ぶ
5. `OritimerDialog` で `MultiGoalSettingAlert` を表示する。閉じたら `_loadMultiGoals()` を呼ぶ

削除ボタンの処理︵最後のゴールのみ表示︶:

1. `NativeGeofenceManager.instance.removeGeofenceById('multiGoal_$number')` でジオフェンスを削除する
2. `appParamNotifier.deleteMultiGoalEntry(number: number)` でエントリを削除する
3. `_loadMultiGoals()` で一覧を更新する

6.5 マルチゴール設定ダイアログ︵MultiGoalSettingAlert︶

ファイル: `lib/screens/components/multi_goal_setting_alert.dart`
Widgetの種別: ConsumerStatefulWidget︵with ControllersMixin︶

ローカル状態:

- `Set<int> _registeredSlots` — 登録済みスロット番号のセット。initState で読み込む
- `ItemScrollController _itemScrollController` — 路線リストのスクロール用
- `TextEditingController _searchController` — 駅名検索フォーム用

画面構成:

```
┌──────────────────────────────────────┐
│  multi goal setting          [設定]    │  ← 設定ボタンで保存・ジオフェンス登録
│ ────────────────────────────────────  │
│  [1][2][3][4][5][6][7][8][9][10]       │  ← 赤=登録済み  黄=選択中スロット
│ ────────────────────────────────────  │
│  [ 駅名を検索...      [×] ] [検索]      │
│ ────────────────────────────────────  │
│  ┌─ 山手線 ──────────────────────┐   │
│  │  ○  東京駅   ● 渋谷駅（選択中）   │  │
│  └──────────────────────────────┘   │
└──────────────────────────────────────┘
```

スロット番号の表示ルール:

- 登録済み（`_registeredSlots.contains(e)`）→ 赤背景（`Colors.red.withValues(alpha: 0.3)`）·選択不可
- 選択中（`appParamState.selectedMultiNumber == e`）→ 黄背景（`Colors.yellowAccent.withValues(alpha: 0.3)`）
- それ以外 → 黒背景（`Colors.black.withValues(alpha: 0.3)`）

設定ボタンの処理:

1. `appParamNotifier.saveMultiGoalEntry()` を呼び出す。`false` が返ったら `error_dialog` を表示して終了する
2. `appParamState.selectedStationName` で駅名を取得し、`tokyoTrainState.tokyoTrainList` から `TokyoStationModel` を探す
3. 見つかった場合:
   - `SharedPreferencesService.saveMultiGoalLocation(number, lat, lng)` で座標を保存する
   - `NativeGeofenceManager.instance.createGeofence(zone, geofenceCallback)` でジオフェンスを登録する（ID: `multiGoal_$number`）
4. `Navigator.pop(context)` でダイアログを閉じる

ジオフェンス設定（マルチゴール登録·復元共通）:

- geofenceId: `'multiGoal_N'`（N はスロット番号 0~9）
- radiusMeters: `1000`
- triggers: `{GeofenceEvent.enter}`

- iOS: `initialTrigger: true`
- Android: `expiration: Duration(days: 7)`, `loiteringDelay: Duration(minutes: 1)`, `notificationResponsiveness: Duration(seconds: 10)`

6.6 SharedPreferencesService

ファイル: `lib/utility/shared_preferences_service.dart`

SharedPreferences への読み書きをまとめたユーティリティクラス。インスタンス化不可。static メソッドのみ。

キー定数:

定数名	値	用途
`kIsSetStation`	`'isSetStation'`	ジオフェンス監視状態
`_kMultiGoalPrefix`	`'multiGoal_'`	マルチゴール駅名。`multiGoal_0` ~ `multiGoal_9`
`_kMultiGoalLatPrefix`	`'multiGoalLat_'`	マルチゴール緯度
`_kMultiGoalLngPrefix`	`'multiGoalLng_'`	マルチゴール経度
`_kRoutePatternPrefix`	`'routePattern_'`	ルートパターン。`routePattern_0` ~ 最大50件

公開メソッド一覧:

メソッド名	引数	
`loadIsSetStation()`	なし	`Future<bool`
`saveIsSetStation(bool flag)`	flag	`Future<void`
`saveMultiGoalEntry({int number, String stationName})`	number, stationName	`Future<void`
`saveMultiGoalLocation({int number, double lat, double lng})`	number, lat, lng	`Future<void`

メソッド名	引数	
`loadMultiGoalLocation({int number})`	number	`Future<({do`
`loadMultiGoalEntry({int number})`	number	`Future<Stri`
`deleteMultiGoalEntry({int number})`	number	`Future<voic`
`clearAllMultiGoalEntries()`	なし	`Future<voic`
`loadAllMultiGoalEntries()`	なし	`Future<Map<`
`saveRoutePattern({String name, List<String> stations})`	name, stations	`Future<voic`
`loadAllRoutePatterns()`	なし	`Future<Map<`
`deleteRoutePattern({int slot})`	slot	`Future<voic`
`saveSelectedStation(String json)`	json	`Future<voic`
`loadSelectedStation()`	なし	`Future<Stri`
`removeSelectedStation()`	なし	`Future<voic`

`saveRoutePattern` の保存形式:

```
{
  "name": "1710480000000",
  "stations": ["高田馬場", "品川", "蒲田"]
}
```

- キー: `routePattern_N` ︵N は 0 から始まる空きスロット番号︶
- 値: 上記 JSON を `jsonEncode()` した文字列

6.7 パターン一覧ダイアログ︵PatternRouteDisplayAlert︶

ファイル: `lib/screens/components/pattern_route_display_alert.dart`
Widgetの種別: ConsumerStatefulWidget︵with ControllersMixin︶

ローカル状態:

- `Map<int, ({String name, List<String> stations})> _patternMap` — スロット番号→パターン情報。initState で `SharedPreferencesService.loadAllRoutePatterns()` から読み込む
- `List<String>? _selectedStations` — 装填中のパターンの駅リスト。CircleAvatar タップ時に設定される。null は未選択を意味する

画面構成:

```
┌──────────────────────────────────────────┐
│  pattern route list                  [設定]  │
│ ──────────────────────────────────────── │
│  ① 高田馬場 → 品川 → 蒲田          [  ]     │
│  ② 渋谷 → 新宿                      [  ]     │
│  ...                                          │
└──────────────────────────────────────────┘
```

- リストが空の場合: `Center(child: Text('登録なし'))` を表示する
- CircleAvatar の背景色: 選択中
  (`appParamState.selectedPatternDispString == dispString`) → 赤
  (`Colors.red.withValues(alpha: 0.3)`) 、それ以外 → 黄
  (`Colors.yellowAccent.withValues(alpha: 0.2)`)

CircleAvatar タップ(装填)の処理:

1. `setState(() => _selectedStations = stations)` でローカル状態を更新する
2. `appParamNotifier.setSelectedPatternDispString(str: dispString)` でハイライト文字列を更新する

「設定」ボタンの処理(`_selectedStations == null` の場合は何もしない):

1. `NativeGeofenceManager.instance.removeAllGeofences()` で既存ジオフェンスをすべて削除する
2. `SharedPreferencesService.clearAllMultiGoalEntries()` で既存マルチゴールエントリをすべて削除する
3. `_selectedStations` の各駅について(インデックス i = 0 ~ stations.length - 1):
   - `SharedPreferencesService.saveMultiGoalEntry(number: i, stationName: stationName)`

で駅名を保存する
    - `tokyoTrainState.tokyoTrainList` を全走査して駅名が一致する `TokyoStationModel` を検索する︵outer ラベル付き二重ループ︶
    - 見つかった場合:
        - `SharedPreferencesService.saveMultiGoalLocation(number: i, lat: lat, lng: lng)` で座標を保存する
        - `NativeGeofenceManager.instance.createGeofence(zone, geofenceCallback)` でジオフェンスを登録する︵例外は無視︶
4. `Navigator.pop(context)` でダイアログを閉じる

削除ボタンの処理:

1. `SharedPreferencesService.deleteRoutePattern(slot: slot)` でパターンを削除する
2. 削除したパターンが選択中だった場合 ︵`appParamState.selectedPatternDispString == dispString`︶:
    - `appParamNotifier.setSelectedPatternDispString(str: '')` でハイライトをクリアする
    - `setState(() => _selectedStations = null)` でローカル選択状態をクリアする
3. `_loadPatterns()` でリストを再読み込みする

ジオフェンス設定︵パターン適用時︶:

- geofenceId: `'multiGoal_N'` ︵N はパターン内の順序インデックス︶
- radiusMeters: `1000`
- triggers: `{GeofenceEvent.enter}`
- iOS: `initialTrigger: true`
- Android: `expiration: Duration(days: 7)`, `loiteringDelay: Duration(minutes: 1)`, `notificationResponsiveness: Duration(seconds: 10)`

---

7. API連携仕様

> **このセクションの目的**：サーバーとの通信の﹁契約﹂を定義する。フロントエンドとバックエンドの開発者が、このセクションを見ればお互いの期待するリクエスト/レスポンスを理解できる。

7.1 共通仕様

項目	値
ベースURL	`http://toyohide.work`
通信方式	HTTP POST
データ形式	JSON
文字コード	UTF-8
エラー時	ステータスコード200以外の場合は例外をスローし`空リストを返す

HttpClient クラス (`data/http/client.dart`):

```
class HttpClient {
  static String get baseUrl => dotenv.env['BASE_URL'] ?? '';

  static Future<http.Response> post({required String path}) async {
    final url = Uri.parse('$baseUrl$path');
    final response = await http.post(url);
    return response;
  }
}
```

APIパス定義 (`data/http/path.dart`):

```
enum APIPath {
  getTokyoTrainStation;

  String get path {
    switch (this) {
      case APIPath.getTokyoTrainStation:
        return '/BrainLog/api/getTokyoTrainStation';
    }
  }
}
```

7.2 エンドポイント詳細

POST /BrainLog/api/getTokyoTrainStation

```
目的:        東京の全路線・駅情報の取得
リクエスト:  ボディなし（POSTメソッドのみ）
レスポンス:  TokyoTrainModel の JSON配列
使用Provider: tokyoTrainProvider
```

レスポンス例:

```
[
  {
    "trainNumber": 1,
    "trainName": "山手線",
    "station": [
      {
        "id": "station_001",
        "stationName": "東京",
        "address": "東京都千代田区丸の内一丁目",
        "lat": 35.6812,
        "lng": 139.7671
      },
      {
        "id": "station_002",
        "stationName": "渋谷",
        "address": "東京都渋谷区道玄坂一丁目",
        "lat": 35.6580,
        "lng": 139.7016
      }
    ]
  },
  {
    "trainNumber": 2,
    "trainName": "中央線",
    "station": [...]
  }
]
```

エラーハンドリング:

```
// tokyoTrainProvider の fetchAllTokyoTrainData() 内
try {
  final response = await HttpClient.post(path: APIPath.getTokyoTrainStation.path);
  if (response.statusCode == 200) {
    final List<dynamic> json = jsonDecode(response.body);
    return json.map((e) => TokyoTrainModel.fromJson(e)).toList();
  } else {
    Utility.errorLog('statusCode: ${response.statusCode}');
    return [];
  }
} catch (e) {
  Utility.errorLog(e.toString());
  return [];
}
```

---

8. ジオフェンス仕様

> **このセクションの目的**：ジオフェンスの設定値とプラットフォーム別の挙動の違いを明確にする。iOSとAndroidでは動作特性が異なるため、ここで整理しておく。

8.1 共通設定

設定項目	値	説明
geofenceId（単体）	`'station_${駅名}'`	単一駅選択のジオフェンス識別子
geofenceId（マルチゴール）	`'multiGoal_N'`（N=0~9）	マルチゴールのジオフェンス識別子
radiusMeters	`1000.0`	ジオフェンスの半径（メートル）
triggers	`[GeofenceEvent.enter]`	入場イベントのみ（退場イベントは対
同時登録数	複数可（単体1件 + マルチゴール最大10件）	マルチゴールは個別に `removeGeofenceById()` で削除する。単体は

設定項目	値	説明
		`removeAllGeofences()` で全削除する

8.2 iOS固有設定

設定項目	値	説明
initialTrigger	`true`	アプリ起動時にすでにジオフェンス圏内にいる場合もイベントを発火する

8.3 Android固有設定

設定項目	値	説明
expiration	`Duration(days: 7)`	ジオフェンスの有効期限︵7日間︶。期限後は自動解除
loiteringDelay	`Duration(minutes: 1)`	圏内に進入してから実際にイベントを発火するまでの
responsiveness	`Duration(seconds: 10)`	OSがジオフェンスイベントをチェックする間隔の目安

> **補足**：Android の `loiteringDelay` は、電車の通過など一瞬だけ圏内に入ったケース（例: 高架から見える駅）の誤検知を防ぐために設定している。1分間滞留して初めてイベントが発火する。

8.4 プラットフォーム別の注意点

事項	iOS	Android
バックグラウンド動作	iOS固有のバックグラウンドモード設定が必要	省電力設定（Dozeモード）
精度	OSのGPSによる。精度は場所・端末による	OSのGPS・Wi-Fi・基地局による複合測位
パーミッション	「常に許可」が必須	`ACCESS_BACKGROUND_LOCA` が必須

8.5 パーミッション要求フロー

```
[🔒 パーミッション要求ボタン をタップ]
        │
        ▼
  位置情報（Fine）を要求
   (permission_handler: Permission.location)
        │
        ├── 拒否 → エラーメッセージ表示。終了
        │
        ▼
  位置情報（常に許可）を要求
   (permission_handler: Permission.locationAlways)
  ※ iOSでは設定アプリに誘導が必要な場合がある
        │
        ├── 拒否 → エラーメッセージ表示。終了
        │
        ▼
  通知パーミッションを要求
   (permission_handler: Permission.notification)
        │
        ├── 拒否 → 通知なし（バイブレーションのみ）で継続
        │
        ▼
  _isPermissionGranted.value = true
   (AppBarの🔒アイコンが黄色に変わる)
```

---

9. ビルド手順

> **このセクションの目的**：開発環境のセットアップ方法を残す゜┐動かない┘を防ぐ゜

9.1 前提条件

```
- Flutter SDK: 3.x 以上
- Dart SDK: 3.x 以上（Flutter同梱のものを使用）
- Android Studio または Xcode（実機テスト用）
```

9.2 環境変数の設定

`.env` ファイルを `assets/` ディレクトリに作成する:

```
BASE_URL=http://toyohide.work
```

9.3 コード生成

モデル︵freezed︶やProvider︵riverpod_generator︶を変更した場合は｀以下のコマンドでコード生成が必要:

```
cd flutter_oritimer
flutter pub run build_runner build --delete-conflicting-outputs
```

> **注意:** `dart run` ではなく `flutter pub run` を使うこと。
> asdf等で管理されたDartは別バージョンの可能性があるため｀Flutter同梱のDartを使う必要がある。

9.4 生成されるファイル

元ファイル	生成ファイル	内容
`model/tokyo_train_model.dart`	`.freezed.dart`, `.g.dart`	freezed + JSON変換コード
`controllers/app_param/app_param.dart`	`.freezed.dart`, `.g.dart`	freezed + Riverpod Provider
`controllers/_get_data/tokyo_train/tokyo_train.dart`	`.freezed.dart`,	freezed +

元ファイル	生成ファイル	内容
	`.g.dart`	Riverpod Provider

9.5 実機ビルドと動作確認

```
cd flutter_oritimer
flutter run            # デバッグビルド＆実行
flutter build apk      # APK生成（Android）
flutter build ipa      # IPA生成（iOS）※要Xcodeと証明書
```

ジオフェンスのテスト方法:

```
Android Emulator:
  1. エミュレータの [Extended controls] → [Location] を開く
  2. 選択した駅の緯度経度を入力し、[SEND] を押す
  3. ジオフェンスの境界（駅座標から1000m離れた地点→1000m以内の地点）を
     Send で変化させてイベントをシミュレートする

実機:
  実際に選択した駅の近くへ移動してテストする
  （エミュレータより信頼性が高い）
```

改訂履歴

版	日付	
1.0	2026-02-28	初版作成
1.1	2026-03-01	ジオフェンス半径500m→1000mに変更（1.1・1.2・8.1・テスト方法）°vibration/shared_preferencesパッケージ追加（2.1）°バイブレーションループ・最大強度
1.2	2026-03-03	選択駅のSharedPreferences永続化を追記（4.3キー表・6.2初期化処理）°_saveS

版	日付	
1.3	2026-03-03	flutter_volume_controller を技術スタックに追加︵2.1︶ °geofenceCallbackに音
1.4	2026-03-07	scrollable_positioned_list を技術スタックに追加︵2.1︶ °駅名検索機能を追加︵ CustomScrollView+SliverList から ScrollablePositionedList.builder に変更︵6.2︶
1.5	2026-03-08	選択駅ジャンプ·同名駅循環ジャンプ機能を追加︵1.2·6.2: _destinationOccurrenc
1.6	2026-03-14	複数目的地登録機能を追加 °SharedPreferencesService ユーティリティクラスを components/·screens/parts/·utility/shared_preferences_service.dart 追加︶ °Ap キー表にマルチゴールキーを追加︵4.3︶ °appParamProvider にマルチゴール操 _multiGoalMap·_loadMultiGoals()·_restoreMultiGoalGeofences()· ﹁複数﹂ボタン
1.7	2026-03-14	ポップアップメニューの全行に×ボタンを表示するよう変更︵6.2 AppBarボタン `isSetStation
1.8	2026-03-15	`geofenceCallback` と `getTrainIndicesForStation` を `home_screen.dart` か `kVibrationIntensities` ︶に抽出︵6.3︶ °ディレクトリ構成に `const/` と `ut`
1.9	2026-03-15	ルートパターン管理機能を追加 °PatternRouteDisplayAlert 画面仕様を追加︵6.7 `clearAllMultiGoalEntries()` · `saveRoutePattern()` · `loadAllRoutePatterns(` フィールドを追加︵4.3︶ °appParamProvider に `setSelectedPatternDispStri` に ﹁パターン﹂ボタン· ﹁パターン登録﹂ボタン仕様を追加︵6.4︶ °ディレクト